

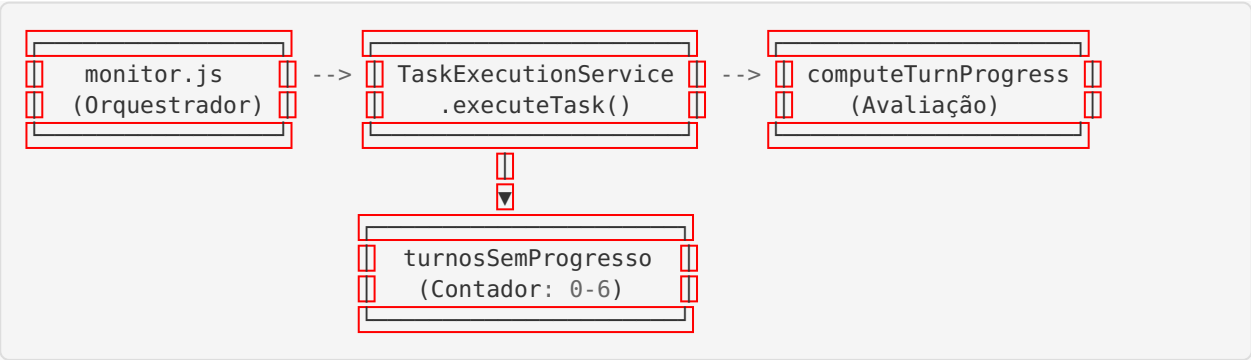
Lógica de Detecção de Mudanças e Ciclos Sem Progresso

Visão Geral

O sistema de monitoramento de tarefas utiliza uma lógica sofisticada para detectar se o agente de IA está fazendo progresso real ou apenas “girando em círculos” sem efetivamente avançar na execução da tarefa. Esta documentação explica em detalhes como essa detecção funciona.

Arquitetura do Sistema

Fluxo Principal



Arquivos Envolvidos

Arquivo	Função
<code>monitor.js</code>	Orquestrador principal que chama <code>TaskExecutionService.executeTask()</code>
<code>src/services/taskExecutionService.js</code>	Contém a lógica completa de detecção de mudanças

Como Funciona o Contador de Ciclos Sem Mudanças

Variáveis Principais

```
// Linha 1628-1629 do taskExecutionService.js
let turnosSemProgresso = 0;
const MAX_TURNOS_SEM_PROGRESSO = 6;
```

Lógica de Incremento/Reset

```
// Linhas 1842-1854 do taskExecutionService.js
if (houveProgresso) {
  turnosSemProgresso = 0; // Reset se houve progresso
} else {
  turnosSemProgresso++; // Incrementa se não houve progresso
  console.log(`⚠️ Turno sem progresso real: ${turnosSemProgresso}/${MAX_TURNOS_SEM_PROGROSSO}`);

  // Adiciona alerta anti-loop ao input do próximo turno
  currentInput += `
[ALERTA ANTI-LOOP]
Você está repetindo ações sem gerar novas evidências úteis.
NÃO repita os mesmos comandos...`;
}
```

O Que Acontece ao Atingir 6 Ciclos

```
// Linhas 1873-1879 do taskExecutionService.js
if (turnosSemProgresso >= MAX_TURNOS_SEM_PROGROSSO) {
  contractResult = {
    contractFulfilled: false,
    executionNotes: `Agente entrou em estagnação: ${MAX_TURNOS_SEM_PROGROSSO} turnos consecutivos sem progresso real.`
  };
  break; // SAI DO LOOP - TAREFA FALHA
}
```

Resultado: A tarefa é marcada como **falha** e o sistema encerra a execução.

CrITÉrios para Determinar se Houve Mudança

Função `computeTurnProgress()`

Esta é a função central que avalia se um turno teve progresso significativo:

```
// Linhas 2058-2089 do taskExecutionService.js
function computeTurnProgress(toolResult = {}, contractResult = {}, cwd = process.cwd()) {
  // 1. Infere arquivos lidos a partir de comandos shell
  const inferred =
    Array.isArray(toolResult.commandsExecuted) && toolResult.commandsExecuted.length
    > 0
    ? inferReadOnlyEvidenceFromCommand(toolResult.commandsExecuted[0], cwd)
    : { filesRead: [], modifiedFiles: [] };

  // 2. Filtra leituras significativas (exclui arquivos efêmeros)
  const meaningfulReads = mergeUniquePaths(
    toolResult.filesRead || [],
    inferred.filesRead || []
  ).filter((file) => !isEphemeralArtifact(file));

  // 3. Filtra mutações significativas (exclui arquivos efêmeros)
  const meaningfulMutations = uniquePaths(
    toolResult.modifiedFiles || []
  ).filter((file) => !isEphemeralArtifact(file));

  // 4. Verifica se foi uma mutação sem efeito
  const noOpMutation = !!toolResult.executionDiagnostics?.noOpMutation;

  return {
    meaningfulReads,
    meaningfulMutations,
    noOpMutation,
    // 5. DECISÃO FINAL: Houve progresso?
    hasMeaningfulProgress:
      !!contractResult.contractFulfilled || // Contrato cumprido SEMPRE é progresso
      (
        !noOpMutation && // Não foi uma mutação vazia
        (
          meaningfulReads.length > 0 || // Leu arquivos novos significativos
          meaningfulMutations.length > 0 // Modificou arquivos significativos
        )
      ),
  };
}
```

Arquivos Efêmeros (Não Contam como Progresso)

Função `isEphemeralArtifact()`

```
// Linhas 1952-1962 do taskExecutionService.js
function isEphemeralArtifact(filePath = '') {
  const base = path.basename(filePath || '');

  return (
    /^terminal-\d+\.log$/i.test(base) || // Logs de terminal
    /^relatorio-\d+\.txt$/i.test(base) // Relatórios
    // NOTA: .done NÃO é mais considerado artefato efêmero
    // porque criar .done é o objetivo final da tarefa
  );
}
```

Por que excluir esses arquivos?

Arquivo Padrão	Razão da Exclusão
<code>terminal-{id}.log</code>	É gerado automaticamente pelo sistema, não pelo agente
<code>relatorio-{id}.txt</code>	Escrever relatório sem fazer alterações reais não é progresso

Importante: O arquivo `.done` **NÃO** é considerado efêmero porque criá-lo é o objetivo final da tarefa.

Mutação Sem Efeito (noOpMutation)

O que é?

Uma “mutação sem efeito” ocorre quando o agente executa um comando que **deveria** modificar arquivos, mas **nenhuma alteração real** acontece.

Como é detectado?

O `CommandExecutor` compara fingerprints dos arquivos antes e depois da execução:

```
// Em commandExecutor.js
// Se um comando como `sed -i` é executado mas o arquivo não muda,
// executionDiagnostics.noOpMutation = true
```

Tratamento

```
// Linhas 1834-1839 do taskExecutionService.js
if (turnProgress.noOpMutation) {
  const noOpFeedback =
    'O último comando mutante executou sem erro no shell, mas não alterou nenhum' +
    'arquivo de fato. ' +
    'Não continue reescrevendo por número de linha. Leia novamente o trecho atual do' +
    'arquivo e use edição por bloco.';

  currentInput = `${currentInput}\n\n[SISTEMA]\n${noOpFeedback}`;
}
```

Detecção de Loops de Ação Repetida

Além do contador de ciclos sem progresso, existe uma detecção de padrões repetitivos:

Assinatura de Turno

```
// Linhas 1857-1861 do taskExecutionService.js
const turnSignature = this.buildTurnSignature(executionResult);
recentActionSignatures.push(turnSignature);
if (recentActionSignatures.length > 60) {
  recentActionSignatures.shift();
}
```

Deteção de Padrão

```
// Linhas 1863-1871 do taskExecutionService.js
const loopInfo = this.detectRepeatedActionLoop(recentActionSignatures, 12, 8);
if (loopInfo.detected) {
  console.log(`❌ Loop detectado: padrão repetido ${loopInfo.repetitions}x`);
  contractResult = {
    contractFulfilled: false,
    executionNotes: `Loop detectado: o agente repetiu o mesmo padrão de ações ${loop-
Info.repetitions} vezes...`
  };
  break;
}
```

Parâmetros:

- `maxPatternSize = 12` : Tamanho máximo do padrão a detectar
- `repetitions = 8` : Número de repetições para considerar loop

Exemplos Práticos

✅ Exemplo 1: Turno COM Progresso

Turno 15:

- Agente executou: `read /src/services/userService.js`
- `toolResult.filesRead = ['/src/services/userService.js']`
- `isEphemeralArtifact('/src/services/userService.js') = false`

- ☐ `meaningfulReads = 1`
- ☐ `hasMeaningfulProgress = true`
- ☐ `turnosSemProgresso = 0` (resetado)

❌ Exemplo 2: Turno SEM Progresso

Turno 18:

- Agente executou: `exec ls -la`
- Nenhum arquivo lido ou modificado

- `meaningfulReads = 0`
- `meaningfulMutations = 0`
- `hasMeaningfulProgress = false`
- `turnosSemProgresso++` (incrementado para 3)

❌ Exemplo 3: Mutação Sem Efeito

Turno 22:

- Agente executou: `exec sed -i 's/foo/foo/' /src/file.js`
- O comando executou sem erro
- Mas o arquivo não mudou (foo -> foo é igual)

- ❌ `noOpMutation = true`
- ❌ `hasMeaningfulProgress = false`
- ❌ `turnosSemProgresso++` (incrementado)
- ❌ Sistema envia feedback especial ao agente

❌ Exemplo 4: Arquivo Efêmero

Turno 25:

- Agente executou: `write relatorio-123.txt "Relatório final..."`
- `toolResult.filesWritten = ['relatorio-123.txt']`
- `isEphemeralArtifact('relatorio-123.txt') = TRUE`

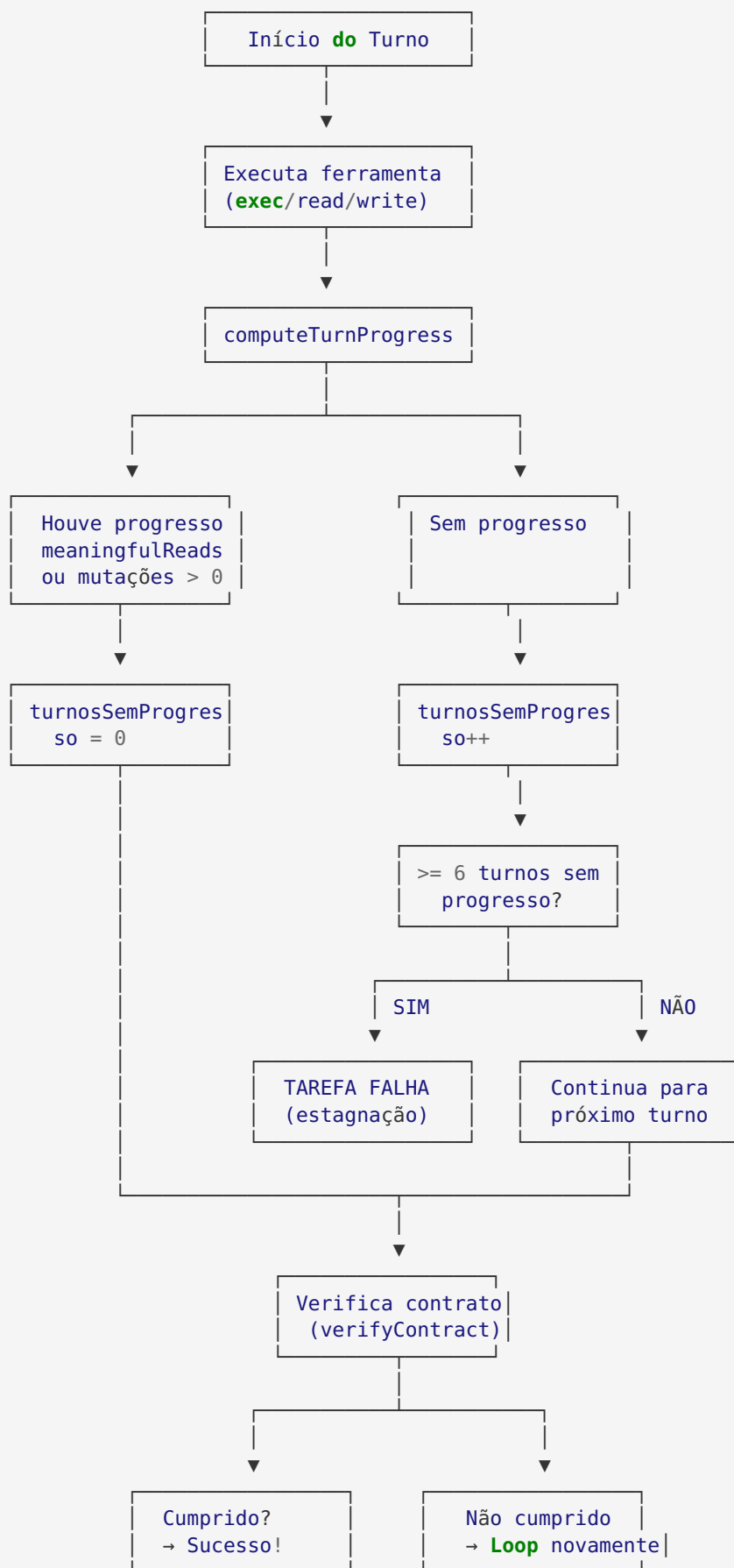
- ❌ `meaningfulMutations = 0` (filtrado)
- ❌ `hasMeaningfulProgress = false`
- ❌ `turnosSemProgresso++` (incrementado)

✅ Exemplo 5: Contrato Cumprido

Turno 30:

- `contractResult.contractFulfilled = true`
 - Mesmo sem leituras ou mutações neste turno específico
- `hasMeaningfulProgress = true` (primeira condição **do OR**)
- `turnosSemProgresso = 0` (resetado)
- Tarefa concluída com sucesso!
-

Fluxograma Completo



Logs de Debug

O sistema gera logs detalhados para acompanhamento:

```
// Linhas 1826-1832 do taskExecutionService.js
console.log(`📊 [PROGRESSO] Turno ${turnos}:`);
console.log(`    contractFulfilled: ${contractResult.contractFulfilled}`);
console.log(`    meaningfulReads: ${turnProgress.meaningfulReads.length}`);
console.log(`    meaningfulMutations: ${turnProgress.meaningfulMutations.length}`);
console.log(`    noOpMutation: ${turnProgress.noOpMutation}`);
console.log(`    hasMeaningfulProgress: ${houveProgresso}`);
console.log(`    turnosSemProgresso: ${turnosSemProgresso}/${MAX_TURNOS_SEM_PROGRESSO}`);
```

Resumo Executivo

Critério	Descrição
Progresso detectado quando	Arquivos não-efêmeros são lidos OU modificados E não é noOpMutation
Progresso NÃO detectado quando	Apenas arquivos efêmeros são tocados OU comandos triviais (ls, pwd) OU mutação sem efeito
Limite de estagnação	6 turnos consecutivos sem progresso
Ação ao atingir limite	Tarefa marcada como falha, execução encerrada
Deteção adicional	Loop de ações repetidas (padrão 8x em 12 ações)

Conclusão

O sistema implementa uma detecção robusta de progresso que:

1. **Diferencia** entre ações reais e ações cosméticas
2. **Ignora** arquivos de infraestrutura (logs, relatórios)
3. **Detecta** mutações que não tiveram efeito real
4. **Fornece feedback** ao agente quando detecta estagnação
5. **Encerra** a tarefa após 6 ciclos sem progresso, evitando loops infinitos
6. **Detecta** padrões repetitivos de ações para evitar loops sutis

Esta abordagem garante que o agente de IA seja efetivamente produtivo e não fique “travado” em comportamentos improdutivos.